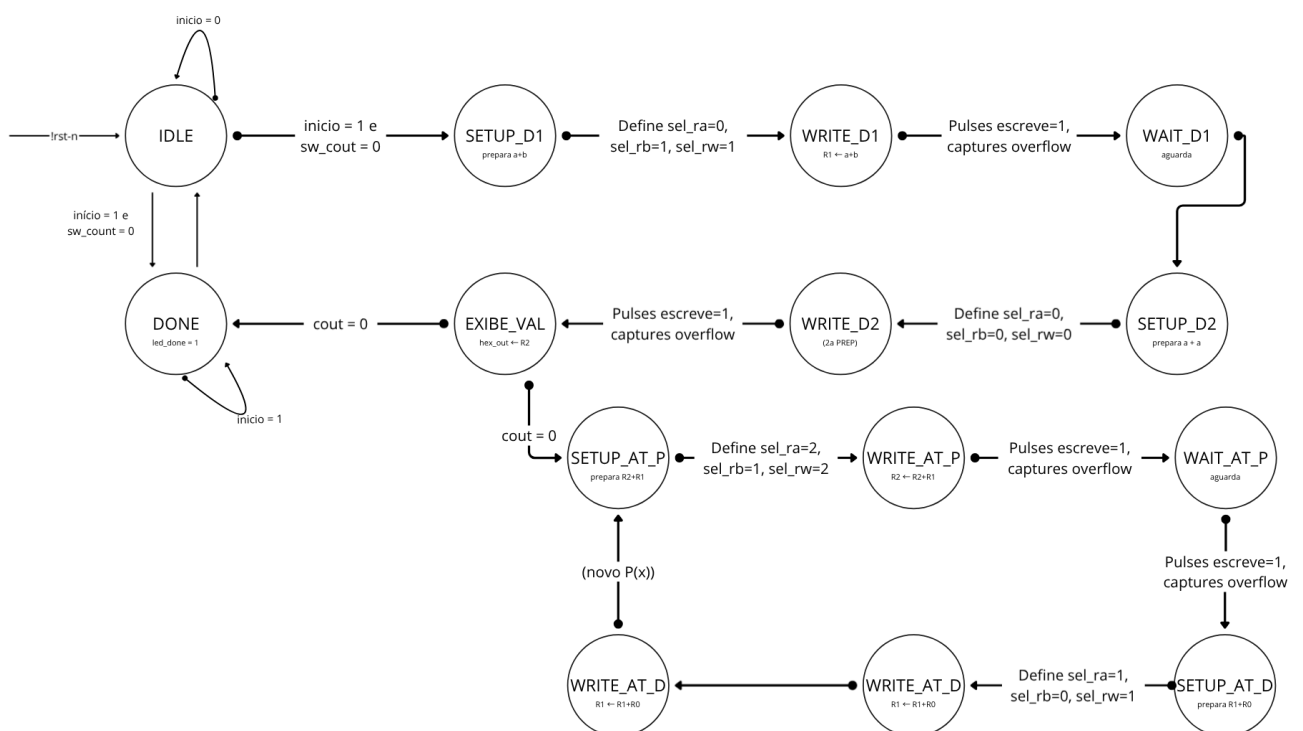


Atividade 2 — Método das Diferenças Finitas (Polinômio de Grau 2)

1. Diagrama de estados FSM



Análise da FSM [fsm_atividade2](#), que controla o datapath (banco de registradores + ULA) fornecido.

Mapeamento de registradores adotado (carga manual antes do [inicio](#)):

Registrador	Conteúdo inicial	Significado
R0	a	coeficiente quadrático de $P(x) = a \cdot x^2 + b \cdot x + c$
R1	b	coeficiente linear
R2	c	termo constante = $P(0)$

Após a Fase 1, R0 passa a guardar a 2ª diferença constante (2a) e R1 passa a guardar a 1ª diferença corrente ($\Delta P(x)$); R2 guarda sempre o valor corrente de P(x).

2. Tabela de Transição de Estados

Estados (registrador de estado, 4 bits): IDLE=0000, SETUP_D1=0001, WRITE_D1=0010, WAIT_D1=0011, SETUP_D2=0100, WRITE_D2=0101, EXIBE_VAL=0110, SETUP_AT_P=0111, WRITE_AT_P=1000, WAIT_AT_P=1001, SETUP_AT_D=1010, WRITE_AT_D=1011, WAIT_AT_D=1100, DONE=1101.

Estado Atual	Condição (entrada)	Próximo Estado	Observação
IDLE	início = 0	IDLE	aguarda partida; overflow_reg é zerado todo ciclo aqui
IDLE	início = 1 e sw_count = 0	DONE	nenhum valor a calcular
IDLE	início = 1 e sw_count > 0	SETUP_D1	count ← sw_count; inicia Fase 1
SETUP_D1	(incondicional)	WRITE_D1	prepara ULA: A=R0(a), B=R1(b)
WRITE_D1	(incondicional)	WAIT_D1	pulso de escrita; se c=1 → overflow_reg ← 1
WAIT_D1	(incondicional)	SETUP_D2	aguarda escrita estabilizar (escreve volta a 0)
SETUP_D2	(incondicional)	WRITE_D2	prepara ULA: A=R0(a), B=R0(a)
WRITE_D2	(incondicional)	EXIBE_VAL	pulso de escrita ($R0 \leftarrow 2a$); se c=1 → overflow
EXIBE_VAL	count = 0	DONE	todos os valores solicitados já exibidos
EXIBE_VAL	count ≠ 0	SETUP_AT_P	count ← count - 1; inicia novo passo da Fase 2
SETUP_AT_P	(incondicional)	WRITE_AT_P	prepara ULA: A=R2, B=R1
WRITE_AT_P	(incondicional)	WAIT_AT_P	pulso de escrita ($R2 \leftarrow R2 + R1$ = novo P(x)); checa overflow
WAIT_AT_P	(incondicional)	SETUP_AT_D	aguarda escrita estabilizar

SETUP_AT_D	(incondicional)	WRITE_AT_D	prepara ULA: $A=R1, B=R0$
WRITE_AT_D	(incondicional)	WAIT_AT_D	pulso de escrita ($R1 \leftarrow R1+R0 =$ nova ΔP); checa overflow
WAIT_AT_D	(incondicional)	EXIBE_VAL	retorna para exibir o novo $P(x)$ calculado
DONE	início = 1	DONE	permanece sinalizando conclusão
DONE	início = 0	IDLE	rearma para nova execução

Apenas três estados são condicionais (IDLE, EXIBE_VAL, DONE); todos os demais são de passagem incondicional, usados para sequenciar corretamente o pulso de escrita síncrona do banco de registradores (padrão SETUP → WRITE → WAIT em quase todas as transições aritméticas).

3. Tabela de Sinais de Controle do Datapath por Estado

Valores padrão (aplicados quando não indicado em contrário pelo estado): **escreve**=0, **sel_ra**=000, **sel_rb**=000, **sel_rw**=000, **sel_op**=011(soma), **mux_w_sel**=0, **ext_w**=0000000000.

mux_w_sel é sempre 0 e **ext_w** é sempre 0 em toda a FSM — o valor escrito no banco vem sempre da saída da ULA (S), nunca de entrada externa, já que toda a carga inicial (a, b, c) é feita manualmente antes do **início**. **sel_op** é sempre **011 (soma)** em todos os estados, conforme exigido pela atividade (método opera só com somas).

Estado	escreve	sel_ra	sel_rb	sel_rw	sel_op	Operação realizada
IDLE	0	000	000	000	011	nenhuma (ocioso)
SETUP_D1	0	000 (R0=a)	001 (R1=b)	001 (R1)	011	combina $A=a, B=b$ na ULA (sem escrever ainda)
WRITE_D1	1	000 (R0=a)	001 (R1=b)	001 (R1)	011	$R1 \leftarrow a + b$ (1ª diferença inicial)
WAIT_D1	0	000	000	000	011	nenhuma (aguarda estabilização)
SETUP_D2	0	000 (R0=a)	000 (R0=a)	000 (R0)	011	combina $A=a, B=a$ na ULA

WRITE_D2	1	000 (R0=a)	000 (R0=a)	000 (R0)	011	$R0 \leftarrow a + a = 2a$ (2ª diferença, constante)
EXIBE_VAL	0	010 (R2)	000	000	011	apenas leitura: $hex_out \leftarrow R2$ (valor corrente de P(x))
SETUP_AT_P	0	010 (R2)	001 (R1)	010 (R2)	011	combina $A=P(x)$, $B=\Delta P(x)$
WRITE_AT_P	1	010 (R2)	001 (R1)	010 (R2)	011	$R2 \leftarrow P(x) + \Delta P(x) = P(x+1)$
WAIT_AT_P	0	000	000	000	011	nenhuma
SETUP_AT_D	0	001 (R1)	000 (R0)	001 (R1)	011	combina $A=\Delta P(x)$, $B=2a$
WRITE_AT_D	1	001 (R1)	000 (R0)	001 (R1)	011	$R1 \leftarrow \Delta P(x) + 2a = \Delta P(x+1)$
WAIT_AT_D	0	000	000	000	011	nenhuma
DONE	0	000	000	000	011	nenhuma; $led_done = 1$

Saídas auxiliares (não conectadas ao datapath, mas relevantes ao sistema):

- hex_out = valor de va no estado corrente; é significativo apenas em **EXIBE_VAL** (mostra $R2 = P(x)$) e em **DONE** (mostra o último valor por herança, já que sel_ra default = $R0$... na prática o display "congela" no último $P(x)$ mostrado, pois o estado **DONE** não força nova leitura).
- $led_overflow = overflow_reg$, atualizado de forma síncrona sempre que $c=1$ em **WRITE_D1**, **WRITE_D2**, **WRITE_AT_P** ou **WRITE_AT_D**; só é zerado ao reentrar em **IDLE**.
- $led_done = 1$ somente em **DONE**.

4. Questões de Análise e Fundamentação Analítica

- a. Como os valores iniciais carregados no banco de registradores são derivados dos coeficientes do polinômio? Demonstre para um exemplo concreto.

Para $P(x) = a \cdot x^2 + b \cdot x + c$, as diferenças finitas de uma função quadrática satisfazem:

- $P(0) = c$
- $\Delta P(0) = P(1) - P(0) = a + b$
- $\Delta^2 P = \Delta P(x+1) - \Delta P(x) = 2a$ (constante, independente de x , pois o grau é 2)

Logo a carga manual no banco de registradores deve ser:

- $R0 \leftarrow a$
- $R1 \leftarrow b$
- $R2 \leftarrow c$

E a própria FSM, na Fase 1, transforma esses valores nas grandezas realmente usadas na recorrência: R0 passa a guardar **2a** (a 2ª diferença constante) e R1 passa a guardar **a+b** (a 1ª diferença $\Delta P(0)$), enquanto R2 permanece como **c = P(0)**, ponto de partida da iteração.

Exemplo numérico: seja $P(x) = 2x^2 + 3x + 5$ ($a=2$, $b=3$, $c=5$).

- Carga inicial: $R0=2$, $R1=3$, $R2=5$
- Fase 1: $R1 \leftarrow 2+3 = 5$ ($\Delta P(0)$); $R0 \leftarrow 2+2 = 4$ ($\Delta^2 P = 2a$)
- Fase 2 (iterando $R2 \leftarrow R2+R1$ e $R1 \leftarrow R1+R0$):

x	P(x) exibido (R2)	$\Delta P(x)$ interno (R1)
0	5	5
1	10	9
2	19	13
3	32	17
4	49	21

Conferindo por avaliação direta: $P(1)=2+3+5=10$, $P(2)=8+6+5=19$, $P(3)=18+9+5=32$, $P(4)=32+12+5=49$ — todos coincidem, validando o mapeamento de registradores.

b. Quantas operações de soma são realizadas a cada novo valor calculado? Compare com a avaliação direta do polinômio e discuta a vantagem do método.

A cada novo valor de $P(x)$ gerado na Fase 2, a FSM realiza exatamente **duas somas de 10 bits** na ULA:

1. **WRITE_AT_P**: $R2 \leftarrow R2 + R1$ (atualiza $P(x)$)
2. **WRITE_AT_D**: $R1 \leftarrow R1 + R0$ (atualiza $\Delta P(x)$ para o próximo passo)

Esse custo é **constante (O(1))**: não depende do valor de x nem do tamanho dos números envolvidos, apenas dos 6 estados de sequenciamento (SETUP/WRITE/WAIT, duas vezes) mais 1 ciclo de exibição — 7 ciclos de clock por valor novo, sempre o mesmo número.

Em contraste, a avaliação direta de $P(x) = a \cdot x^2 + b \cdot x + c$ para um x arbitrário exige, no mínimo (forma de Horner: $P(x) = (a \cdot x + b) \cdot x + c$), **2 multiplicações + 2 somas** — ou seja, o dobro de operações aritméticas, e ainda dependeria de manter x em um registrador e incrementá-lo a cada passo. Além disso, a multiplicação no datapath fornecido produz um resultado intermediário de 20 bits (**result_mul**) antes de truncar para 10 bits, o que aumenta o risco e a complexidade da detecção de overflow.

A vantagem do método das diferenças finitas é, portanto: (i) elimina completamente a necessidade do operador de multiplicação durante a fase iterativa — usa só `sel_op = 011`; (ii) tem custo fixo e previsível por valor gerado, não crescente com x ; (iii) simplifica a FSM e reduz a área/complexidade do hardware necessário, ao preço de uma fase inicial de preparo (duas somas extras, únicas, no início).

O método das diferenças finitas introduz erro acumulado? Justifique considerando a aritmética inteira de 10 bits utilizada.

Em aritmética inteira exata, o método **não introduz nenhum erro de arredondamento ou truncamento**: como a , b , c e todas as diferenças ($a+b$, $2a$, etc.) são inteiros, e a única operação usada é a soma inteira, cada $P(x)$ calculado pela recorrência é matematicamente idêntico ao valor obtido por avaliação direta do polinômio — não há aproximação envolvida (diferente do que ocorreria em ponto flutuante).

O risco real está no **overflow do registrador de 10 bits** (faixa 0–1023). O somador do datapath é de 11 bits (`result_wide`), e o resultado de 10 bits é obtido por truncamento (`s = result_wide[9:0]`), com o bit de carry sinalizado em `c`. A FSM apenas *registra* esse overflow em `led_overflow`, mas **não corrige nem interrompe o cálculo** — o valor truncado (incorreto) continua sendo gravado no registrador.

Isso tem uma consequência importante e específica do método incremental: como cada novo $P(x)$ e cada nova $\Delta P(x)$ são calculados **a partir do valor anterior** ($R1$ e $R2$ realimentam a si mesmos), um overflow ocorrido em um determinado passo **se propaga e se acumula em todos os passos subsequentes** — diferente de uma avaliação direta e independente de $P(x)$ a cada x , na qual um erro em um ponto não contaminaria o cálculo dos pontos seguintes. Ou seja: o método das diferenças finitas é exato sob aritmética ilimitada, mas é **mais vulnerável à propagação de erro de overflow** do que a avaliação direta, justamente por sua natureza recursiva/incremental.

LINK YOUTUBE: <https://youtube.com/shorts/HxEfZnSiayw>